

SimDuoReal: Object-Centric Bimanual Policy for Emergent Dexterity

Anonymous Authors

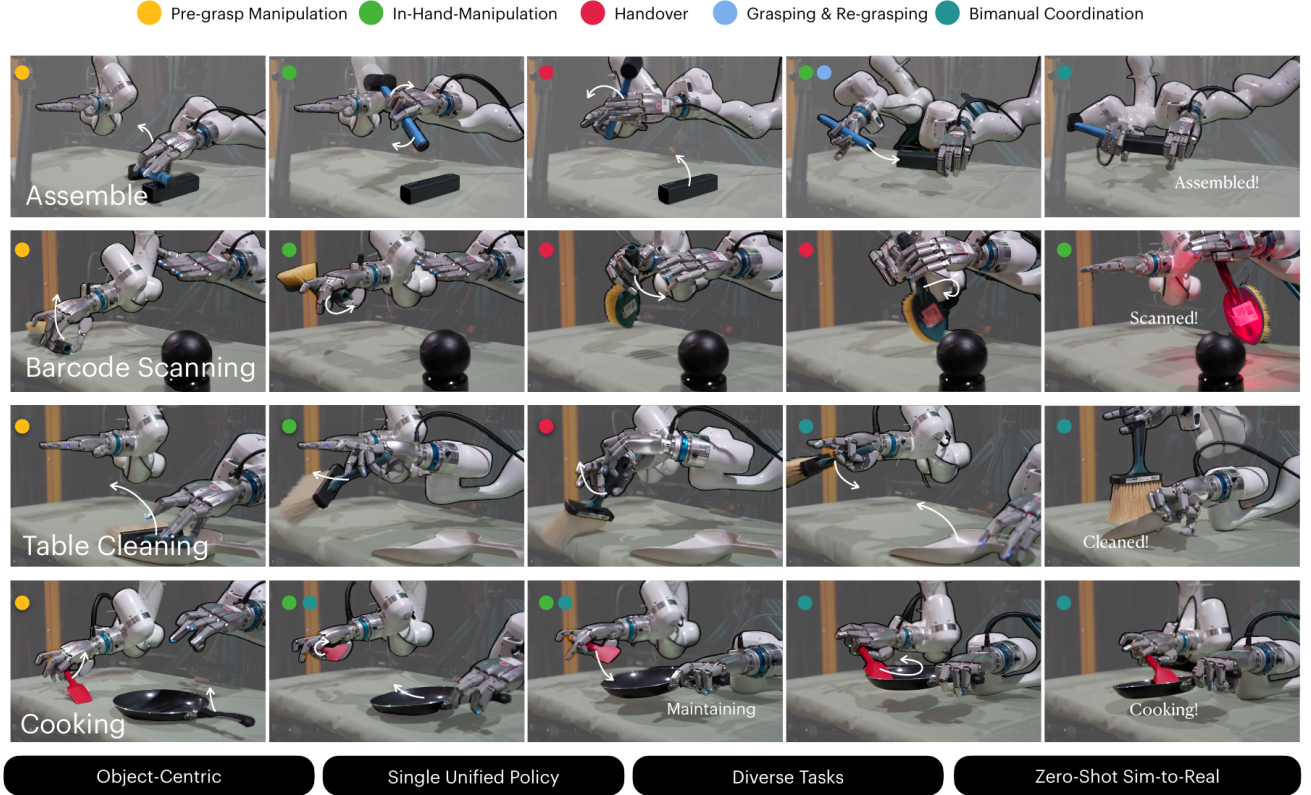


Fig. 1: SimDuoReal is a single object-centric policy that controls two dexterous arm-hand systems from a stream of desired object configurations. The same policy, transferred zero-shot from simulation, performs assembly, barcode scanning, table cleaning, and cooking on real hardware. Markers denote the behaviors that emerge along each rollout: pre-grasp manipulation (orange), in-hand manipulation (green), handover (red), grasping and re-grasping (blue), and bimanual coordination (teal).

Abstract—Bimanual dexterous manipulation requires coordinated arm and hand control, yet specifying such coordination manually is challenging and often task-specific. In this work, we explore whether complex bimanual dexterity can emerge from a simple object-centric learning objective. We train reinforcement learning policies to control two dexterous arm-hand systems by optimizing the manipulated object’s trajectory toward a desired goal pose. We find that this object-centric formulation encourages the policy to discover grasping, regrasping, in-hand adjustment, and handover behaviors as a means to accomplish the task. Our experiments show that object-centric reinforcement learning can produce surprisingly rich bimanual behaviors without explicit skill decomposition or motion retargeting. On real hardware, the same policy succeeds in 61 of 80 trials (76.3%) across four tasks with unseen objects and no task-specific fine-tuning. In simulated barcode scanning, our policy achieves 85.0% success, compared with 60.4% for two single-arm policies without handover. These results suggest that this objective provides a simple and scalable interface for learning general bimanual dexterity, shifting the burden from manually designing robot coordination to specifying what should happen to the object. Website: simduoreal.github.io.

I. INTRODUCTION

Bimanual dexterous manipulation systems have the potential to perform a wide range of everyday tasks: picking up an object, reorienting it to expose a desired face, and passing it between hands. Existing systems have demonstrated these capabilities on individual tasks [1]–[3]. However, specifying these behaviors is challenging because the robot must decide not only how to move the object, but also which hand to use and when to pass it between hands. Existing methods often specify this coordination through task-specific rewards or skill sequences. These specifications are difficult to reuse across tasks, so extending the system to new behaviors requires repeated engineering.

Object-centric formulations sidestep part of this problem by specifying tasks purely in terms of the object’s desired state, leaving the robot free to discover how to bring it about. For example, SimToolReal [1] trains a single-arm policy to move procedurally generated tool-like objects to random 6-DoF goal poses and transfers zero-shot to unseen real tools. Tasks can

then be specified as sequences of object goal poses without object- or task-specific training. Extending this approach to two hands over a larger workspace, however, poses a learning challenge. For goals beyond either hand’s reach, the policy must discover how to transfer the object before it can complete the goal. RL can instead settle for partial progress with one hand without discovering the handover needed to reach the goal.

The straightforward approach is to add task-specific rewards that shape hand selection and handover. This can guide exploration, but requires specifying how the hands should coordinate for each behavior. New behaviors may then require new reward terms, weakening the benefit of a reusable object-goal interface.

In this work, we extend object-centric task specification to bimanual dexterous manipulation by designing *training environments* rather than hand-craft separate *rewards* for each behavior. We vary the distribution of object goals and constrain each hand’s workspace under a shared object-centric reward. Bimanual behaviors emerge during training, without explicitly programming how the hands should pick up, reorient, or pass the object (Fig. 1).

More specifically, we define a reachable workspace for each hand and train two single-arm experts for object control within their respective workspaces. Furthermore, we train a bimanual expert for cross-workspace transfer (Sec. III-B). All three share the same object-centric representation and reward, while their training tasks only differ in goal sampling, episode initialization and termination. For single-arm training, the object starts on the table and the goals stay within that arm’s reach. The hand learns to pick up the object, reorient it, and change its grasp as the goals change. For handover training, one hand starts holding the object, and the goal is sampled from the other hand’s workspace, forcing the holding hand to pass the object to the other hand to reach the goal. A goal is complete only when the receiving hand contacts the specified surface region on the object and the giving hand releases the object. As a result, the giver learns to adjust the object before passing it, exposing this region for a *transfer grasp* rather than a *hold grasp* (Fig. 5). We distill the experts into one policy using behavior cloning and RL, after fine-tuning each expert from states reached by preceding experts. During distillation, the policy is trained on both individual expert behaviors and their transitions (Sec. III-C).

Our policy can be prompted with objects: we specify how the objects should move, and the policy controls the arms and hands to carry out that motion without task-specific training. An operator can provide these prompts by moving virtual objects in VR. We also extract object trajectories from RGB-D human demonstrations using FoundationPose [4] and use them as pose waypoints. We evaluate barcode scanning in simulation with unseen object shapes. On real hardware, we use object-level teleoperation to test unseen objects on four tasks: barcode scanning, peg insertion, cleaning, and cooking (see Fig. 1). We also compare expert-first and end-to-end training and ablate expert refinement, the distillation loss, and the training scenes (Sec. IV).

Contributions.

- We design training environments that use object-goal sampling and workspace constraints to learn single-arm control and bimanual handover under a shared object-centric reward, rather than separate rewards for each behavior.
- We show that pickup, in-hand adjustment, regrasping, and handover emerge without hand-designed motion sequences. Before handover, the giver learns *transfer grasps* that expose the object region the receiver needs to grasp.
- We prompt *one* bimanual policy with object motion to execute new tasks without task-specific training. We demonstrate this with unseen real objects in barcode scanning, peg insertion, cleaning, and cooking.

II. RELATED WORK

Bimanual Dexterous Manipulation. Bimanual systems with multifingered hands have demonstrated coordinated behaviors such as twisting lids, tool use, and passing objects between hands [5]–[9]. Reinforcement learning systems specify each behavior through task-specific rewards or staged sub-goals [5]–[7], while imitation systems specify it through demonstrations collected per task [8]–[11]. In both cases the specification includes which hand acts and when the object changes hands, so a new behavior requires new reward terms or new demonstrations. We instead use a single object-centric reward and vary only the goal distribution and the workspace constraints used during training.

Object-Centric Task Specification. A second line of work specifies manipulation through object-centric task specification. In-hand reorientation reaches 6-DoF object pose goals with a single hand [2], [12]–[15], and recent methods use object trajectories as the learning signal for dexterous policies [16]–[18]. SimToolReal [1] trains a single-arm policy to move procedurally generated objects to random goal poses and transfers zero-shot to unseen real tools, so tasks become sequences of object goal poses. These formulations assume one hand and one workspace. With two hands, pose error alone does not distinguish a handover from a single-hand carry, and the carry is available whenever the goal is reachable; SimDuoReal makes the transfer the only strategy that reaches the goal.

Composing Specialists into One Policy. Training specialists and compressing them into one network is common practice: teacher–student distillation folds privileged or terrain-specific policies into a single deployable one [19]–[22], and skill-chaining work observes that a policy degrades on the initiation states its predecessor actually produces, and regularizes terminal states to close that gap [23], [24]. These pipelines still define each skill by its own reward or reference motion. Our experts differ only in goal sampling, so one reward supervises both the refinement of each expert on its predecessors’ terminal states and the reinforcement learning term used during distillation.

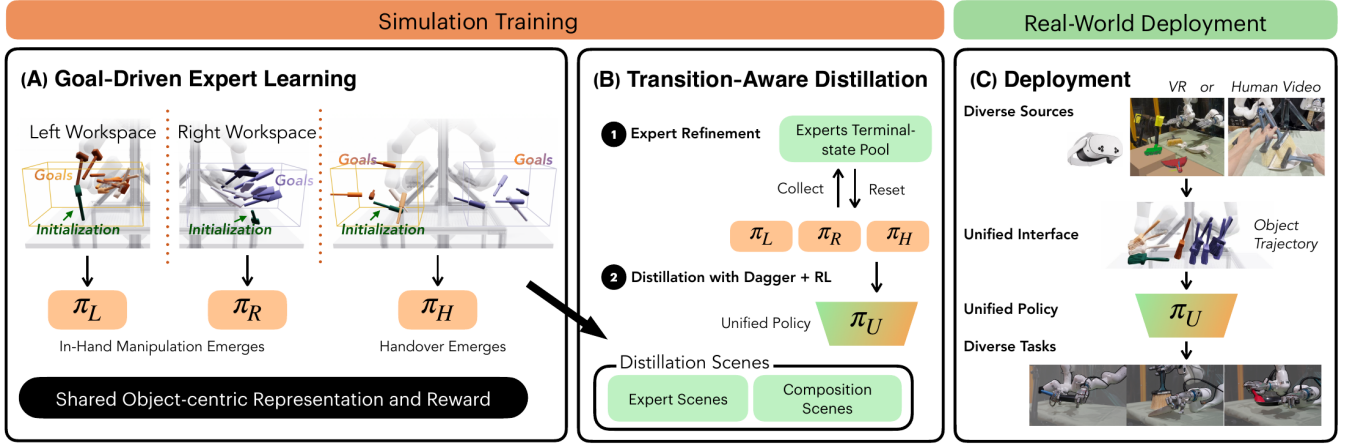


Fig. 2: System overview (A) Three experts are trained in simulation under a shared object-centric representation and reward, differing only in their goal distributions: the left and right experts π_L and π_R learn grasping with in-hand manipulation from goals inside one arm’s workspace, and the handover expert π_H learns handover from goals that shift from the giver’s workspace to the receiver’s. (B) The experts are refined from a pool of each other’s terminal states, then distilled into a unified policy π_U with Dagger and RL in scenes that contain both individual expert behaviors and their transitions. (C) At deployment, object trajectories from diverse sources such as VR teleoperation and videos serve as the unified interface, and the single policy π_U executes diverse real-world tasks.

III. METHOD

We propose SimDuoReal, a single bimanual policy for manipulating one or two objects toward desired 6-DoF poses (Fig. 2). Through this common goal interface, SimDuoReal acquires dexterous behaviors such as grasping, in-hand adjustment and handover, which can be composed to execute downstream tasks such as coordinated tool use. Our key idea is that these behaviors can emerge from a *shared object-centric* formulation by varying only the goal distributions during training. We first train three workspace-conditioned experts for within-workspace control and cross-workspace transfer (Secs. III-B). We then refine these experts at transition boundaries and distill them into a unified policy using DAGger and reinforcement learning in scenes spanning individual expert regimes and their sequential or concurrent composition (Secs. III-C). At deployment, the unified policy tracks object trajectories obtained from VR or human video (Secs. III-D), enabling diverse zero-shot bimanual tool manipulation.

A. Problem Formulation

We consider the problem of learning a bimanual dexterous policy that manipulate up to two objects to desired 6-DoF pose sequences. At control step t , let $\mathcal{J}_t \subseteq \{1, 2\}$ index the active objects. For each object $j \in \mathcal{J}_t$, $\mathbf{T}_t^j, \hat{\mathbf{T}}_t^j \in \mathbf{SE}(3)$ denote its current and goal poses. The policy observes the robot proprioception $\mathbf{s}_t$, the current and goal object poses, and object-centric geometric representation $\mathbf{r}_t$ introduced in Secs. III-B:

$$\mathbf{a}_t = \pi_\theta(\mathbf{s}_t, \{\mathbf{T}_t^j, \hat{\mathbf{T}}_t^j, \mathbf{r}_t\}_{j \in \mathcal{J}_t}) \in \mathbb{R}^{58}, \quad (1)$$

where $\mathbf{a}_t$ specifies joint-position targets for both robot arms and hands.

A task is specified by an object-pose trajectory $\tau^j = \{\hat{\mathbf{T}}_k^j\}_{k=1}^{K_j}$ for each active object. The policy is conditioned on the current pose goal and advances to the next one

once the pose error d_t^j falls below a threshold ϵ . For cross-workspace object transfer, this condition additionally requires the receiving hand to establish contact and the giving hand to release the object. For two-object tasks, the two trajectories can advance either independently or synchronously.

B. Learning Workspace Experts through Goal Sampling

We define a Left Workspace and a Right Workspace to indicate the reachable space for each hand, with an overlapping zone in the middle (see Fig. 2). We train two single-arm experts $\mathbf{a}_t^k = \pi_k(\mathbf{o}_t^k) \in \mathbb{R}^{29}, k \in \{L, R\}$, for object control within their respective workspaces and one bimanual expert $\mathbf{a}_t^H = \pi_H(\mathbf{o}_t) \in \mathbb{R}^{58}$ for cross-workspace transfer. All three share the same object-centric representation and reward, while their training tasks differ in goal sampling and episode initialization and termination. **Grasping, in-hand adjustment, and handover** therefore *emerge* from object-pose reaching without skill-specific motion references or rewards.

1) Goal-Sampling Regimes:

a) *Within-workspace control per arm*: The episode begins with an un-grasped object on the table. An initial goal position is sampled within the arm’s reachable workspace $\mathcal{W}_k \subset \mathbb{R}^3$, and subsequent goals are generated by translations and rotations while remaining within $\mathcal{W}_k$. Repeatedly reaching these goals requires grasping and sustained object control while also producing *in-hand adjustment and regrasping* as goals change.

b) *Cross-Workspace Transfer*: The bimanual expert is trained from states in which one hand, designated as the giver initially holds the object. The goal-sampling region then shifts from the giver’s workspace into the receiver’s workspace excluding the overlapping zone. We sample N_{shift} goals in the cross-over setting and N_{dst} further goals in the destination workspace. We train with both directions by exchanging giver and receiver roles. During expert training we set $N_{\text{shift}} = N_{\text{dst}} = 1$ to focus learning on the handover itself.

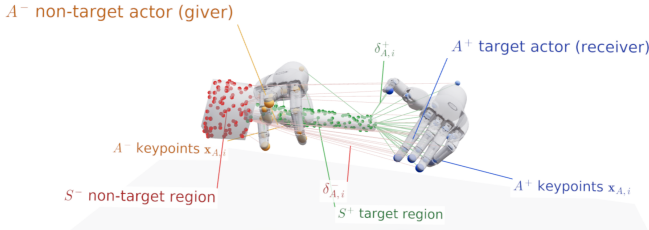


Fig. 3: The object-centric representation This figure shows the representation for the manipulated object during handover. The object’s surface splits into the target contact region S^+ (handle, green) and the non-target contact region S^- (head, red); nearest-region displacements $\delta_{A,i}^+$ (green) and $\delta_{A,i}^-$ (red) relate the keypoints $\mathbf{x}_{A,i}$ of the target actor A^+ (receiver, blue) and the non-target actor A^- (giver, orange) to both regions.

A cross-workspace goal is completed only after the receiver establishes contact and the giver releases the object. Dropping the object or violating the workspace constraints terminates the episode. This goal distribution induces *handover* and *preparatory object adjustment* without specifying either behavior explicitly.

2) Shared Object-Centric Formulation:

a) Role-Conditioned Object-Centric Representation:

The policy must reason about both where an object should move and how actor in contact should behave. For each active object j , we assign a target actor A_j^+ and a non-target actor A_j^- . The target actor should establish contact with the object, whereas the non-target actor should separate from it. Thus, A_j^+ denotes the active arm during within-workspace control and the receiving arm during transfer; A_j^- denotes the table in the former and the giving arm in the latter.

Similarly, we divide the object surface into target and non-target contact regions S_j^+ and S_j^- . Let $X_A = \{\mathbf{x}_{A,i}\}_{i=1}^{n_A}$ denote the keypoints of actor A and $S_j^a = \{\mathbf{y}_{j,m}^a\}_{m=1}^{M_j^a}$ denote the surface points in region $a \in \{+, -\}$, where n_A is the number of keypoints on actor A and M_j^a is the number of points sampled on object j ’s surface region a . For each actor keypoint, we compute its 3D displacement vector to the nearest point in each region as in [25]:

$$\delta_{A,i}^{j,a} = \mathbf{y}^* - \mathbf{x}_{A,i}, \quad \mathbf{y}^* = \arg \min_{\mathbf{y} \in S_j^a} \|\mathbf{y} - \mathbf{x}_{A,i}\|. \quad (2)$$

We concatenate these displacements over all active objects, actors, and regions to form $\mathbf{r}_t$, jointly representing two objects in the bimanual scene. This representation provides the policy with a common geometric description of approaching, maintaining, and releasing contact across all experts.

b) *Shared Object-Centric Reward:* Our objective is to reach the desired object poses while satisfying the assigned contact roles. We build on the reward formulation of SimToolReal [1], adapting its definitions to the bimanual setting. All specialists optimize

$$r_t = \left[\sum_{j \in \mathcal{J}_t} \left(r_t^{\text{pose},j} + r_t^{\text{con}+,j} + r_t^{\text{con}-,j} + r_t^{\text{succ},j} \right) - r_t^{\text{vel}} \right] r_t^{\text{pen}}. \quad (3)$$

The pose and target-contact terms reward progress rather than absolute proximity. We measure pose error using corresponding current and target object keypoints $\mathbf{p}_{t,\ell}^j$ and $\hat{\mathbf{p}}_{t,\ell}^j$, and define

$$\begin{aligned} d_t^{\text{pose},j} &= \max_{\ell} \left\| \mathbf{p}_{t,\ell}^j - \hat{\mathbf{p}}_{t,\ell}^j \right\|_2, \\ r_t^{\text{pose},j} &= \lambda_{\text{pose}} \text{clip} \left(d_{t,\min}^{\text{pose},j} - d_t^{\text{pose},j}, 0, c \right), \\ d_{t+1,\min}^{\text{pose},j} &= \min \left(d_{t,\min}^{\text{pose},j}, d_t^{\text{pose},j} \right). \end{aligned} \quad (4)$$

Here, $d_{t,\min}^{\text{pose},j}$ stores the smallest pose error reached under the current goal and is reset when a new goal is assigned. The keypoint distance jointly captures translation and rotation error, while rewarding only improvements prevents the policy from accumulating reward by oscillating near the goal.

The target-contact term $r_t^{\text{con}+,j}$, follows the same progress-based formulation, replacing the pose error with the distance between the target actor and its designated object region $d_{A,i}^{j,+} = \sum_{i=1}^{n_A} \left\| \delta_{A,i}^{j,+} \right\|_2$. The non-target-contact term $r_t^{\text{con}-,j}$ encourages A_j^- to be away from the object based on $d_{A,i}^{j,-} = \min_{i=1,\dots,n_A} \left\| \delta_{A,i}^{j,-} \right\|_2$ and provides an additional bonus once target actor make contact and non-target actor breaks contact. Finally, $r_t^{\text{succ},j}$ rewards goal completion, r_t^{vel} regularizes motion, and r_t^{pen} penalizes unsafe configurations.

3) *RL Training Details:* We train each expert using EPO [26] with an asymmetric critic. The actor receives only observations available during real-world deployment, while the critic additionally receives privileged simulator state. Following SimToolReal [1], we train on primitive objects with randomized geometry and inertial properties, and adopt its success tolerance curriculum and sim-to-real domain randomization. Specifically, we randomize robot and object dynamics, inject observation noise, and apply external force and torque perturbations to the objects.

C. Distilling Workspace Experts into a Unified Policy

The three experts cover complementary goal regimes but do not directly compose, as each is trained only on its own state distribution. We first refine the experts on transition states generated by preceding experts, then distill them into a unified policy with DAgger and reinforcement learning on both expert and composition scenes.

1) *Transition-Aware Expert Refinement:* We collect the terminal states reached by each expert into an end-state pool. For each target expert, we randomly initialize training episodes from the end states of its valid predecessor experts and continue training under its original goal distribution and reward. This adapts each expert to states encountered at behavior transitions. The refined experts are then used as teachers for distillation.

2) *Unified-Policy Distillation:* We distill the refined experts into a unified bimanual policy π_U using DAgger [27] and reinforcement learning. The unified policy uses the same object-centric observations and action space as the cross-workspace expert π_H . We train π_U on three scene distributions

that expose it to cross-workspace expert behaviors and their deployment-style compositions.

a) Training Scenes: At each episode reset, we sample a cross-workspace expert scene with probability 0.4, an end-to-end cross-workspace scene with probability 0.4, or a concurrent dual-object scene with probability 0.2. The first contains no inter-skill transition, whereas the latter two require within-episode expert transitions.

Cross-workspace expert scenes reproduce the training setting of π_H (Sec. III-B.1.b), providing direct supervision for transfer behavior.

End-to-end cross-workspace scenes begin with an ungrasped object on the table and a goal sequence follows the cross-workspace schedule of Sec. III-B.1, with N_{shift} sampled uniformly from $\{3, \dots, 7\}$ at each reset and $N_{\text{dst}} = 2$. The student must therefore compose source-side pickup and manipulation, cross-workspace transfer, and destination-side manipulation.

Concurrent dual-object scenes require the two arms to manipulate separate objects simultaneously, with one object assigned to each workspace. We query the left- and right-workspace experts simultaneously and combine their corresponding arm actions to construct the teacher action. Each object’s goal advances independently upon completion.

b) Teacher Routing: Teacher supervision is determined by the current and target workspaces of each active object j . If $\mathbf{T}_t^j$ and $\hat{\mathbf{T}}_t^j$ lie in the same workspace, we query the corresponding within-workspace expert, π_L or π_R ; otherwise, we query the cross-workspace expert π_H . Thus, an end-to-end cross-workspace episode naturally transitions from the source-side expert to π_H , and then to the destination-side expert. For concurrent dual-object scenes, each object is routed independently and the resulting arm-specific teacher actions are combined. The router is used only to generate DAgger supervision and is not exposed to π_U .

c) Distillation Objective: During student rollouts, DAgger queries the routed teachers on states visited by π_U . We jointly optimize

$$\mathcal{L} = w_{\text{DAgger}} \mathcal{L}_{\text{DAgger}} + w_{\text{rl}} \mathcal{L}_{\text{rl}}, \quad (5)$$

where $\mathcal{L}_{\text{DAgger}}$ matches the unified policy’s actions to teacher actions, and $\mathcal{L}_{\text{rl}}$ is the policy-gradient loss under the shared reward from Sec. III-B. The imitation term preserves the experts’ established behaviors, while reinforcement learning allows the unified policy to recover from its own errors and improve behavior composition. The loss weights follow the scheduling strategy of [28].

D. Task Generalization through Object-Trajectory Commands

At deployment, a task is specified by one or two object-pose trajectories $\tau^j = \{\hat{\mathbf{T}}_k^j\}_{k=1}^{K_j}$. For waypoint-based trajectories, the policy tracks the active goal until it satisfies the completion condition in Sec. III-A, and then advances to the next waypoint. For two-object tasks, the trajectories can advance independently or synchronously: independent advancement supports concurrent object manipulation, while synchronous

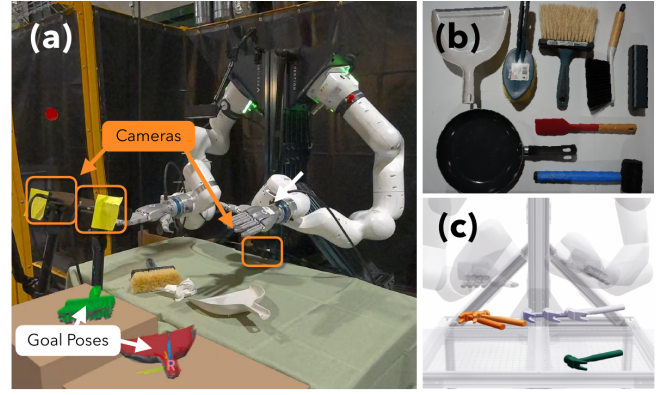


Fig. 4: Experimental setup (a) Real-world setup from the VR view. Three calibrated RGB-D cameras (orange) surround the workspace, and the human controls the object goal poses, shown as rendered object copies, which are streamed to the policy. (b) The real-world object set. (c) An example rule-based object trajectory for the simulated barcode-scanning benchmark.

advancement preserves paired motions such as alignment and insertion.

We obtain object trajectories from two sources. **1) VR:** An operator moves virtual copies of the objects using a Meta Quest 3, and the latest desired poses are streamed to the policy at 60 Hz as online commands. **2) Human Video:** We estimate the object trajectory from a human demonstration using FoundationPose [4] and downsample it into pose waypoints.

In both cases, tasks are specified solely through desired object motion. This object-level interface enables zero-shot execution of diverse tasks, without task-specific fine-tuning or robot-level commands and the unified policy autonomously realizes each trajectory through behaviors that emerged during training, including pre-grasp manipulation, grasping, in-hand adjustment and handover.

IV. EXPERIMENT

A. Experimental Setup

We evaluate SimDuoReal on four real-world tasks and conduct controlled simulation experiments (Fig. 4). The policy runs at 60 Hz and is used across all tasks without task-specific fine-tuning. Objects are unseen during training (Fig. 4b) and initialized ungrasped at randomized initial poses.

Hardware setup We use two Franka Research 3 arms with 22-DoF Sharpa Wave hands. Three RGB-D ZED cameras are calibrated to a shared world frame and jointly cover the bimanual workspace (Fig. 4a).

Object assets and perception stack We obtain rigid object meshes either using AR Code¹ on an iPhone or from RGB videos using SAM3D [29], following SimToolReal [1]. To provide closed-loop object-state feedback, we develop a multi-view perception stack that tracks the poses of two objects at 35 Hz. We run FoundationPose [4] and Cutie [30] on each camera. Per-view pose estimates are transformed into a shared world frame, filtered by observed-to-rendered depth consistency, reinitialized from reliable peer estimate when

¹<https://ar-code.com/>

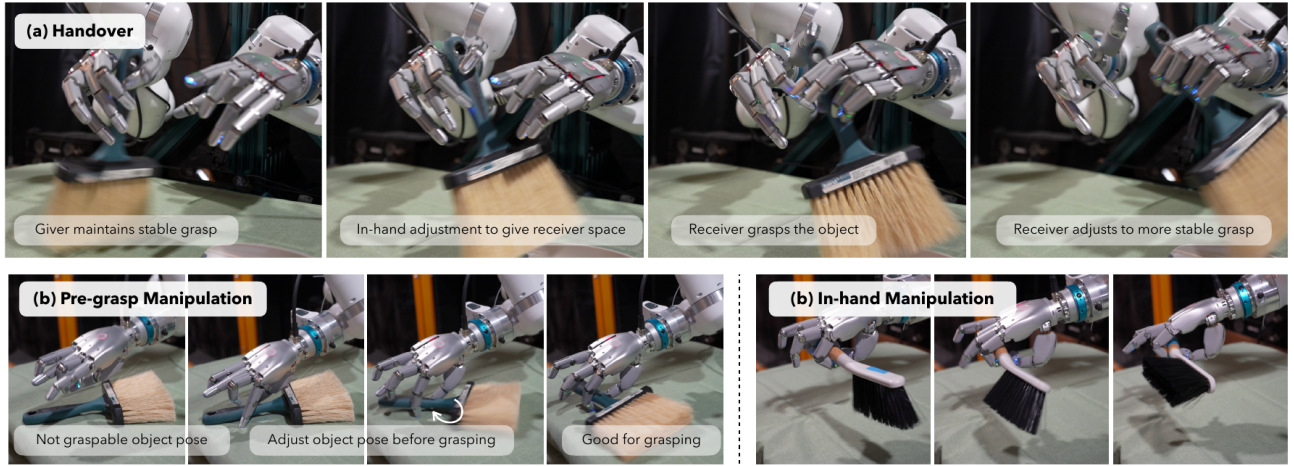


Fig. 5: Emergent behaviors on real hardware (a) Handover: the giver holds a stable grasp, adjusts the brush in-hand to expose a grasable region for the receiver, and releases once the receiver has grasped and settled into a more stable grasp. (b) Pre-grasp manipulation: when the object rests in a pose that is not graspable, the hand first pushes it into a graspable pose. (c) In-hand manipulation: the hand reorients the brush within the grasp to track subsequent object goals. None of these behaviors is specified explicitly; the policy discovers them from object-pose goals alone.

tracking fails, and fuse them using Fixed Lag Smoothing (FLS) over a $SE(3)$ pose graph [15]. The policy uses the latest pose estimation at each control step.

VR interface For real-world experiments, we use a Meta Quest 3 to provide online object-pose goals at 60 Hz (Fig. 4a).

B. Real-World Evaluation

1) Tasks and Evaluation Protocol: We evaluate our unified SimDuoReal policy on four real-world tasks without task-specific fine-tuning. Together, these tasks cover complementary dimensions of bimanual dexterity, including object handover, coordinated manipulation of two objects, and precise object alignment. We conduct 20 trials per task using unseen objects initialized at randomized poses. When both objects initialized in same workspace, handover is required.

Barcode Scanning The object is initialized in either the scanner-side or opposite workspace. The robot must present three distinct brushes to a fixed scanner with its barcode facing the sensor. The task require precise alignment between barcode and scanner. A trial succeeds when the scanner detects the barcode within the time limit.

Peg insertion A peg and a holed block are initialized independently in either workspace or the same workspace. The robot must coordinate both hands to align and insert the peg. Success requires reaching the specified insertion depth.

Cleaning A brush and a dustpan are initialized independently in either workspace or the same workspace. The robot must acquire and transfer the brush before performing a coordinated sweeping motion while maintaining stable grasps and tool alignment. Success requires completing the prescribed sweep without dropping either tool.

Cooking A spatula and a frying pan are initialized in separate workspaces. The robot must stabilize the pan with one hand while executing a stirring motion with the spatula in the other. Success requires completing the prescribed motion without dropping either tool.

TABLE I: Real-world results on four tasks combining complementary bimanual capabilities. The same SimDuoReal policy is used for all tasks without task-specific fine-tuning. Success is reported as the number of successful trials out of 20.

Task	Handover	Two-Object Coordination	Precision	SR
Barcode scanning	✓		✓	15/20
Peg insertion	✓	✓	✓	15/20
Cleaning	✓	✓		16/20
Cooking		✓		15/20
Average (%)				76.25

2) Results: As shown in Table I, SimDuoReal succeeds in 61 of 80 real-world trials, corresponding to an overall success rate of 76.3%. Performance remains consistent across the four tasks: 15/20 for barcode scanning, 15/20 for peg insertion, 16/20 for cleaning, and 15/20 for cooking. Together, these tasks require different combinations of handover, simultaneous two-object coordination, and precise alignment.

Fig. 5 illustrates the emergent dexterity exhibited by SimDuoReal. During handover, the giver reorients the object to expose an accessible region, the receiver establishes a stable grasp, and the giver releases. The policy also performs pre-grasp manipulation to reposition objects from ungraspable configurations, and in-hand manipulation to reach poses unavailable through arm motion alone because of constraints from the table or the other arm. These arm and hand behaviors are not specified by the operator, but arise autonomously as SimDuoReal tracks the commanded object motion.

C. Simulation Analysis and Ablation Study

We use barcode scanning as a controlled simulation benchmark (Fig. 4c) that follows rule-based trajectories that interpolates position and orientation linearly, because successful execution requires grasping an initially ungrasped

TABLE II: **Simulation results on barcode scanning.** We compare full SimDuoReal policy with baselines and distillation variants. Bold marks the best result per metric.

Method	SR (%) $\uparrow$	Prog. (%) $\uparrow$	Time (s) $\downarrow$
Modular baselines			
2 $\times$ SimToolReal [1] (no handover)	60.4	79.6	9.2 $\pm$ 4.9
Chained expert teachers			
SimDuoReal w/o transition-aware refinement	79.4	88.1	9.1 $\pm$ 4.4
SimDuoReal teacher	84.3	89.8	8.7 $\pm$ 4.0
Unified student policy			
SimDuoReal w/o RL loss (DAGger only)	81.6	89.5	9.0 $\pm$ 4.2
SimDuoReal w/ composition scenes only	68.3	84.8	8.8 $\pm$ 4.4
SimDuoReal	85.0	90.8	8.6 $\pm$ 3.9

object, reorienting it toward the scanner, transferring it to scanner workspace when necessary, and precisely reaching the final scan pose. We evaluate each method over 3,072 envs each with distinct unseen primitive objects and randomized initial poses across bimanual workspace. Each env contains $K = 7$ task waypoints. We report three metrics: *success rate* (SR), the percentage of episodes in which the policy reaches the final task goal; *task progress* (Prog.), the percentage of waypoints reached in each episode, averaged over all episodes; and *completion time* (Time), the elapsed simulator time from episode initialization to the final goal, reported as mean and standard deviation over successful episodes.

1) *Why train workspace expert separately:* SimDuoReal separately trains three policies for left-workspace manipulation, right-workspace manipulation, and cross-workspace manipulation before unifying them. We compare with an end-to-end baseline that trains a single policy from scratch on the combined distribution of all three regimes. Fig. 6 compares the two training strategies under the same compute budget. End-to-end training becomes particularly difficult when all required behaviors must be learned jointly from scratch. Moreover, extending the skill set requires retraining on the expanded joint distribution, whereas the expert-based approach can train new skills independently and effectively integrate them into the unified policy through distillation.

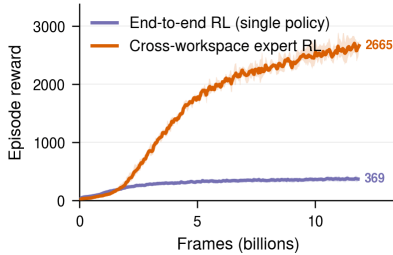


Fig. 6: Expert training vs. End-to-End training. Episode reward against environment frames for a single policy trained end-to-end on the full task and for handover skill training, each with 3 seeds. Both use the same algorithm, environment count, and initial-state distribution.

2) *Why do we need handover:* We compare with 2 $\times$ SimToolReal, using the same left- and right-workspace expert used in our pipeline, but without the handover expert. For cross-workspace episodes, the baseline uses an indirect place-and-regrasp strategy: one arm lifts the object and places it in the shared central region, after which the other arm attempts to regrasp it and continue toward the scan pose. In contrast, SimDuoReal directly coordinates the two hands to transfer the object without an intermediate placement. Adding the handover expert, by chaining all three using a workspace-based routing rule, even without transition-aware refinement (SimDuoReal w/o transition-aware refinement), raises success to 79.4% and progress from 79.6% to 88.1%, while reducing completion time.

3) *Why is transition-aware expert refinement needed:* Although rule-based chaining selects the appropriate expert, it does not ensure that the succeeding expert is prepared for the state produced by its predecessor. In particular, the hand configuration produced by one expert may be poorly matched to the preferred manipulation configuration of the next. We therefore refine each expert on terminal states collected from its predecessors. This refinement increases success from 79.4% to 84.3% and progress from 88.1% to 89.8% showing grasp transition. Without refinement, the succeeding expert can often recover through additional in-hand manipulation or re-grasping, but at the cost of redundant motion and a higher risk of dropping the object. After refinement, the policy performs more stable handovers and more directly transitions toward grasp configurations suitable for subsequent within-workspace manipulation.

4) *How do we distill the experts into a unified policy:* Direct DAGger distillation reaches 81.6% success, below the 84.3% achieved by the refined expert teachers. While DAGger preserves the individual expert behaviors, it does not expose the student to errors accumulated under its own rollouts, leading primarily to failures during transition behaviors. Conversely, training only on composition scenes reduces success to 68.3%, with failures dominated by degraded handover performance. This shows that composition scenes alone do not preserve the transfer behavior learned in cross-workspace expert scenes. Our full procedure therefore combines DAGger and RL on both expert and composition scenes, preserving handover while learning end-to-end and concurrent execution. It reaches 85.0% success, matching the refined expert teachers with a single unified policy.

V. CONCLUSION

We presented SimDuoReal, a bimanual policy that learns dexterous arm–hand coordination from a shared object-centric objective rather than behavior-specific rewards. By varying only the goal-sampling regime under one object-centric representation and reward, we train two within-workspace experts and one cross-workspace expert in which grasping, in-hand adjustment, regrasping, and handover—including the *transfer grasps* that expose the region the receiver needs—*emerge* without prescribed motion sequences. We then distill them into a unified policy that is prompted purely

with object motion from VR or human video. The single policy succeeds in 61 of 80 real-world trials (76.3%) on unseen objects across handover, two-object coordination, and precise alignment, and reaches 85.0% in simulation while matching its refined teachers, suggesting that object-centric task specification offers a simple and scalable interface that shifts the burden from designing how the hands should coordinate to specifying what should happen to the object. Limitations remain: SimDuoReal assumes accurate object meshes and multi-view pose tracking, its ordered 6-DoF goal interface does not directly express force- or timing-dependent behaviors—extending the formulation to richer contact specifications, and autonomous goal generation is a promising direction for future work.

ACKNOWLEDGMENT AND AI-USE DISCLOSURE

OpenAI Codex was used for minor manuscript editing and code refinement. All scientific content and results were developed and verified by the authors.

REFERENCES

- [1] K. Kedia, T. G. W. Lum, J. Bohg, and C. K. Liu, “Simtoolreal: An object-centric policy for zero-shot dexterous tool manipulation,” *arXiv preprint arXiv:2602.16863*, 2026.
- [2] OpenAI, M. Andrychowicz, B. Baker, M. Chociej, R. Jozefowicz, B. McGrew, J. Pachocki, A. Petron, M. Plappert, G. Powell, A. Ray, J. Schneider, S. Sidor, J. Tobin, P. Welinder, L. Weng, and W. Zaremba, “Learning dexterous in-hand manipulation,” *arXiv preprint arXiv:1808.00177*, 2018.
- [3] D. Frau-Alfaro, J. Castaño Amorós, S. Puente, P. Gil, and R. Calandra, “Learning dexterous object handover,” *arXiv preprint arXiv:2506.16822*, 2025.
- [4] B. Wen, W. Yang, J. Kautz, and S. Birchfield, “FoundationPose: Unified 6D pose estimation and tracking of novel objects,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 17 868–17 879.
- [5] Y. Chen, T. Wu, S. Wang, X. Feng, J. Jiang, Z. Lu, S. McAleer, H. Dong, S.-C. Zhu, and Y. Yang, “Towards human-level bimanual dexterous manipulation with reinforcement learning,” in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., 2022.
- [6] T. Lin, Z.-H. Yin, H. Qi, P. Abbeel, and J. Malik, “Twisting lids off with two hands,” *Conference on Robot Learning (CoRL)*, 2024.
- [7] A. Petrenko, A. Allshire, G. State, A. Handa, and V. Makoviychuk, “Dexptb: Scaling up dexterous manipulation for hand-arm systems with population based training,” *Robotics: Science and Systems (RSS)*, 2023.
- [8] T. Lin, Y. Zhang, Q. Li, H. Qi, B. Yi, S. Levine, and J. Malik, “Learning visuotactile skills with two multifingered hands,” *arXiv:2404.16823*, 2024.
- [9] K. Shaw, Y. Li, J. Yang, M. K. Srirama, R. Liu, H. Xiong, R. Mendonca, and D. Pathak, “Bimanual dexterity for complex tasks,” in *8th Annual Conference on Robot Learning*, 2024.
- [10] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” *arXiv preprint arXiv:2304.13705*, 2023.
- [11] Z. Jiang, Y. Xie, K. Lin, Z. Xu, W. Wan, A. Mandlekar, L. Fan, and Y. Zhu, “Dexmimicgen: Automated data generation for bimanual dexterous manipulation via imitation learning,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
- [12] A. Handa, A. Allshire, V. Makoviychuk, A. Petrenko, R. Singh, J. Liu, D. Makoviichuk, K. Van Wyk, A. Zhurkevich, B. Sundaralingam *et al.*, “Dextreme: Transfer of agile in-hand manipulation from simulation to reality,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 5977–5984.
- [13] T. Chen, J. Xu, and P. Agrawal, “A system for general in-hand object re-orientation,” in *Conference on robot learning*. PMLR, 2022, pp. 297–307.
- [14] Z.-H. Yin, B. Huang, Y. Qin, Q. Chen, and X. Wang, “Rotating without seeing: Towards in-hand dexterity through touch,” *arXiv preprint arXiv:2303.10880*, 2023.
- [15] R. Chen, M. Mukadam, M. Kaess, T. Wu, F. R. Hogan, J. Malik, and A. Sharma, “Ptld: Sim-to-real privileged tactile latent distillation for dexterous manipulation,” *arXiv preprint arXiv:2603.04531*, 2026.
- [16] Y. Chen, C. Wang, Y. Yang, and C. K. Liu, “Object-centric dexterous manipulation from human motion data,” *arXiv preprint arXiv:2411.04005*, 2024.
- [17] X. Liu, J. Adalibieke, Q. Han, Y. Qin, and L. Yi, “DexTrack: Towards generalizable neural tracking control for dexterous manipulation from human references,” in *International Conference on Learning Representations*, 2025.
- [18] T. G. W. Lum, O. Y. Lee, C. K. Liu, and J. Bohg, “Crossing the human-robot embodiment gap with sim-to-real rl using one human demonstration,” 2025. [Online]. Available: <https://arxiv.org/abs/2504.12609>
- [19] A. A. Rusu, S. G. Colmenarejo, C. Gulcehre, G. Desjardins, J. Kirkpatrick, R. Pascanu, V. Mnih, K. Kavukcuoglu, and R. Hadsell, “Policy distillation,” in *International Conference on Learning Representations*, 2016.
- [20] S. Ross, G. J. Gordon, and J. A. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.
- [21] Z. Zhuang, Z. Fu, J. Wang, C. Atkeson, S. Schwaertfeger, C. Finn, and H. Zhao, “Robot parkour learning,” in *Conference on Robot Learning (CoRL)*, 2023.
- [22] X. Cheng, K. Shi, A. Agarwal, and D. Pathak, “Extreme parkour with legged robots,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [23] Y. Lee, J. J. Lim, A. Anandkumar, and Y. Zhu, “Adversarial skill chaining for long-horizon robot manipulation via terminal state regularization,” in *Conference on Robot Learning (CoRL)*, 2021.
- [24] Y. Chen, C. Wang, L. Fei-Fei, and C. K. Liu, “Sequential dexterity: Chaining dexterous policies for long-horizon manipulation,” in *Conference on Robot Learning (CoRL)*, 2023.
- [25] H. Zhang, S. Christen, Z. Fan, O. Hilliges, and J. Song, “GraspXL: Generating grasping motions for diverse objects at scale,” in *European Conference on Computer Vision (ECCV)*, 2024.
- [26] J. Wang, Y. Su, A. Gupta, and D. Pathak, “Evolutionary policy optimization,” *arXiv preprint arXiv:2503.19037*, 2025.
- [27] S. Ross, G. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proceedings of the fourteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings, 2011, pp. 627–635.
- [28] Z. Wu, X. Huang, L. Yang, Y. Zhang, K. Sreenath, X. Chen, P. Abbeel, R. Duan, A. Kanazawa, C. Sferrazza *et al.*, “Perceptive humanoid parkour: Chaining dynamic human skills via motion matching,” *arXiv preprint arXiv:2602.15827*, vol. 1, 2026.
- [29] X. Chen, F.-J. Chu, P. Gleize, K. J. Liang, A. Sax, H. Tang, W. Wang, M. Guo, T. Hardin, X. Li *et al.*, “Sam 3d: 3dfy anything in images,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026, pp. 7220–7232.
- [30] H. K. Cheng, S. W. Oh, B. Price, J.-Y. Lee, and A. Schwing, “Putting the object back into video object segmentation,” in *arXiv*, 2023.